

Interface with the user space
M2 SETI-B4 Embedded Linux

Maria Mushtaq
Maria.mushtaq@telecom-paris.fr
2022-2023

Institut Mines-Télécom

Plan

- Introduction
- Classic recall functions
- Data transfer to/from user space
- Frameworks
- Links between the different data structures used
- Conclusion

Classification of devices

- Seen from the user space

  - Character devices (char device): basic unit of transfer = one character (one byte), interaction with standard system calls (open, read, write, ioctl...) on a special file (usually in /dev)

  - Block devices : basic unit of a transfer = data block, also accessible through a special file

  - Network devices : applications can manipulate them through the network stack (socket)

- The majority of the devices are seen as character devices, so we will study them later

Special Files

- The interaction between an application and a character or block device takes place through a special file identified by

  - A mode (character or block) - A major number

  - A minor number

- The list of major/minor numbers is available in Documentation/admin-guide/devices.txt
- These files are traditionally placed in the /dev directory (but it is not mandatory)
- They are created either by calling the mknod command (manually, by a script, or by a daemon like udev), or automatically by the kernel (devtmpfs)

File Operations

- When a driver reacts to the detection of a supported device, it announces to the kernel that it supports a new triplet (mode, major, minor) and declares a set of callback functions
- When an application makes a system call (e.g. read) on a special file, the kernel will call one of the callback functions of the driver associated with the concerned device by passing it the arguments transmitted by the application
- The callback function called in the driver will be responsible for answering the application's request

Struct File_Operations

<include/linux/fs.h>

```
struct file_operations {
  struct module *owner;
  loff_t (*llseek) (struct file *, loff_t, int);
  ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
  ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
  ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
  ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
  int (*iterate) (struct file *, struct dir_context *);
  unsigned int (*poll) (struct file *, struct poll_table_struct *);
  long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
  long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
  int (*mmap) (struct file *, struct vm_area_struct *);
  int (*mremap)(struct file *, struct vm_area_struct *);
  int (*open) (struct inode *, struct file *);
  int (*flush) (struct file *, fl_owner_t id);
  int (*release) (struct inode *, struct file *);
  int (*fsync) (struct file *, loff_t, loff_t, int datasync);
  int (*aio_fsync) (struct kiocb *, int datasync);
  int (*fasync) (int, struct file *, int);
  int (*lock) (struct file *, int, struct file_lock *);
  ssize_t (*sendpage) (struct file *, struct page *, int, size_t, loff_t *, int);
  unsigned long (*get_unmapped_area)(struct file *, unsigned long, unsigned long,
  int (*check_flags)(int);
```

Struct File_Operations

- This structure groups the callback functions provided by the device driver
- This structure is passed by the driver to the kernel when a new device is detected
- Some fields can be set to NULL because the kernel provides a default behavior for most of these functions

Plan

- Introduction
- Classic recall functions
- Data transfer to/from user space
- Frameworks
- Links between the different data structures used
- Conclusion

Error Code

- System calls (e.g. those used to perform operations on files) may fail (permissions problem, non-existent file, etc.)
- This failure is indicated to the user space by a negative integer representing an error code
- At the level of the standard library, this error code is then stored in the variable errno and the function called to make the system call will return -1
- These error codes are defined at kernel level in include/linux/errno.h

Error Code (Extract)
Attention, Positive Integer!

```
#define EPERM            1      /* Operation not permitted */
#define ENOENT           2      /* No such file or directory */
#define ESRCH            3      /* No such process */
#define EINTR            4      /* Interrupted system call */
#define EIO              5      /* I/O error */
#define ENXIO            6      /* No such device or address */
#define E2BIG            7      /* Argument list too long */
#define ENOEXEC          8      /* Exec format error */
#define EBADF            9      /* Bad file number */
#define ECHILD          10      /* No child processes */
#define EAGAIN          11      /* Try again */
#define ENOMEM          12      /* Out of memory */
#define EACCES          13      /* Permission denied */
#define EFAULT          14      /* Bad address */
#define ENOTBLK         15      /* Block device required */
#define EBUSY           16      /* Device or resource busy */
#define EEXIST          17      /* File exists */
#define EXDEV           18      /* Cross-device link */
#define ENODEV          19      /* No such device */
#define ENOTDIR         20      /* Not a directory */
#define EISDIR          21      /* Is a directory */
#define EINVAL          22      /* Invalid argument */
#define ENFILE          23      /* File table overflow */
#define EMFILE          24      /* Too many open files */
/* ... */
```

Open

```
int foo_open(struct inode *inode, struct file *file);
```

- Called by the kernel when an application makes an open system call on the special file representing one of the devices managed by the driver
- Arguments
  - struct inode *inode : represents uniquely the special file on which the system call has been made
  - struct file *file : represents the opened file (several instances of this structure can exist for a single inode if it is opened by several applications or by the same application several times). This structure is passed later to each function (example : read...). It contains some important information (see next slide)

Open
Struct File (Extract)

- fmode_t f_mode : File open for reading (FMODE_READ) or writing (FMODE_WRITE)
- unsigned int f_flags : Various flags, the most important of which is O_NONBLOCK which is set when the application asks for non-blocking file operations
- const struct file_operations *f_op : Pointer to the structure containing the operations for this file
- void *private_data : Pointer that your driver can use to store information. They will then be accessible via this field each time an operation is called for this file

Open

```
int foo_open(struct inode *inode, struct file *file);
```

- Returned value : 0 to indicate that the opening was successful, a negative value to indicate an error (example - EPERM)
- This function may not be defined in the struct file_operations. In this case the kernel considers that the opening of the file by an application will always succeed (except for permission problems)

release

```
int foo_release(struct inode *, struct file *);
```

- Called when the file is closed
- Same arguments as open
- Can be undefined, in which case closing will always succeed

read

ssize_t foo_read(**struct** file *file, **char** __user *buf, size_t count, loff_t *f_pos);

- Called during a read (read system call)
- If not defined, the read will fail with the error -EINVAL
- Arguments

  - struct file *file : The structure representing the file

  - char __user *buf: The buffer, in user space, to fill with data from the device

  - size_t count : The maximum number of bytes to read

  - loff_t *f_pos : Pointer to the current position in the file, to be updated

read

ssize_t foo_read(**struct** file *file, **char** __user *buf, size_t count, loff_t *f_pos);

- Reads at most count bytes from the device and writes them to the buffer in user space buf
- Returns the actual number of bytes written to buf (which may be less than count), 0 to indicate the end of the file or a negative number to indicate an error

read

```
ssize_t foo_read(struct file *file, char __user *buf, size_t count, loff_t *f_pos);
```

- In the normal case, the function must block until at least one byte is available and as soon as it is, return the available bytes right away (even if there are less than count)
- If the O_NONBLOCK flag is set in file->f_flags, if no data is available, the function must return -EAGAIN right away (without blocking) or, if data is available, return it right away (even if it is less than count)

read

```
ssize_t foo_read(struct file *file, char __user *buf, size_t count, loff_t *f_pos);
```

- The use of the buffer in user space buf will be explained later
- In the case of devices, the notion of current position in the f_pos file often has no meaning

  - In a classical file, the read call (as well as write, seek, etc.) modifies this current position, hence this argument

write

```
ssize_t foo_write(struct file *file,
                  const char __user *buf ,
                  size_t count, loff_t *f_pos);
```

- Reads at most count bytes from the buffer in buf user space buffer, sends them to the device
- Returns the actual number of bytes transmitted (which may be less than count), or a negative number to indicate an error

write

```
ssize_t foo_write(struct file *file,
const char __user *buf ,
size_t count, loff_t *f_pos);
```

- In the normal case, the function should block until at least one byte has been written (transmitted to the device) and as soon as this is the case, return the number of transmitted bytes immediately (even if there are fewer than count)
- If the O_NONBLOCK flag is set in file->f_flags, if it is not possible to transmit data to the device immediately, the function must return -EAGAIN immediately (without blocking)

ioctl

```
long foo_unlocked_ioctl(struct file *file,
unsigned int cmd ,
unsigned long arg);
```

- The read/write metaphor is appropriate for transferring data between a device and an application, but it does not easily allow an application to modify or retrieve the configuration of a device (change the speed of a serial port, etc.).
- This function, called after the ioctl system call, allows to exchange information that are not "useful" data between the driver and the application

ioctl

long foo_unlocked_ioctl(**struct** file *file, **unsigned int** cmd, **unsigned long** arg);

- The application provides to the ioctl system call an integer that identifies the operation to be performed (cmd argument) and possibly another argument, integer or pointer (arg argument)
- The meaning of cmd and arg is specific to each driver
- Documentation/driver-api/ioctl.rst contains additional important information about ioctl, including the choice of values for cmd and compat_ioctl

Plan

- Introduction
- Classic recall functions
- Data transfer to/from user space
- Frameworks
- Links between the different data structures used
- Conclusion

Application/core Transfer

- The buffer whose address is provided to the read and write functions is located in the user space part of the application address space. The code running in the kernel space (which concerns the device driver) does not have the right to access this buffer directly (by dereferencing the pointer directly or by using functions like memcpy)

  - Impossible on some architectures

  - Potential security problem (the memory referenced by the pointer may be intentionally or unintentionally invalid, which may lead to a paging error in the kernel, which is forbidden, or worse)

__user

- The __user macro decorates pointers containing addresses that point to the user space
- Depending on the kernel configuration, gcc uses it and checks that no dereferencing of these pointers is done

Transfer Function
<asm/uaccess.h> et <linux/uaccess.h>

- get_user(x, p) : the variable x in kernel space receives the value pointed by the pointer p from user space
- put_user(x, p) : the content of the variable x in kernel space is copied to the address pointed by the pointer p in user space
- unsigned long copy_to_user(void __user *to, const void *from, unsigned long n)
- unsigned long copy_from_user(void *to, const void __user *from, unsigned long n)

    - Transfer n bytes from/to the user space

Plan

- Introduction
- Classic recall functions
- Data transfer to/from user space
- Frameworks
- Links between the different data structures used
- Conclusion

Frameworks

- Frameworks exist to unify and streamline the interface between applications and drivers for similar devices
- Depending on the framework, the abstraction is more or less important
- Examples

  - Input framework: includes many input devices (keyboard, mouse, joystick...).

    Provides an important abstraction based on events

  - Framework misc : very light, just simplifies the management of special files

Framework misc

- In the following, we will describe a little more the very light misc Framework
- Simplifies the management of special files
- This is the first framework you will use to interface between your accelerometer driver and the user space (we could also have used the input framework)

Framework misc

```
struct miscdevice {
  int minor;
  const char *name;
  const struct file_operations *fops;
  struct list_head list;
  struct device *parent;
  struct device *this_device;
  const struct attribute_group **groups;
  const char *nodename;
  umode_t mode;
};
```

Framework misc
struct miscdevice

- Describes a driver using the misc framework Main fields

  - minor : the desired minor number for the device (the major number will automatically be the one of misc devices, i.e. 130) or MISC_DYNAMIC_MINOR to get one dynamically

  - name : the name of the device, used to create more or less automatically the corresponding special file in /dev

  - fops : pointer to the struct file_operations which declares the callback functions used to respond to read, write, etc. operations

  - parent : the associated device

Framework misc
Registration

- A driver registers with the misc framework using the int misc_register(struct miscdevice *misc) function
  - The call to this function is traditionally made in the probe function of the driver When the device is no longer present, remember to call the function void misc_deregister (struct miscdevice *misc)
  - The call to this function is traditionally made in the remove function

Plan

- Introduction
- Classic recall functions
- Data transfer to/from user space
- Frameworks
- Links between the different data structures used
- Conclusion

Problem Statement

- Let's take the case of an I2C device driver using the misc framework
- Each physical device managed by the driver is represented by the struct i2c_client

  - The functions used to perform an I2C transfer (i2c_master_send and i2c_master_recv for example), i.e. communicate with the device to transfer data for example, need this struct i2c_client

Problem Statement

- The driver will also need to store information for each physical device present that it manages (e.g. current state, associated miscdevice struct, etc.).

  - Often this structure is called foo_device where foo is the name of the device (ex. adxl345_device)

  - It is instantiated dynamically (via a call to kmalloc), by the probe function, for each new supported device

Problem Statement

- Finally, when an operation is performed on the special file representing one of the supported physical devices, the kernel will call the corresponding callback function and pass the struct file structure representing the file
- The following problem arises

  - In the function foo_read(struct file *file, char __user *buf, size_t count, loff_t *f_pos), how to find the struct i2c_client structure (necessary to make I2C transfers) or the struct foo_device structure (which we may need)?

Summary of the situation

```
struct i2c_client client {
   /* ... */
   struct device dev;
   /* ... */
}
```

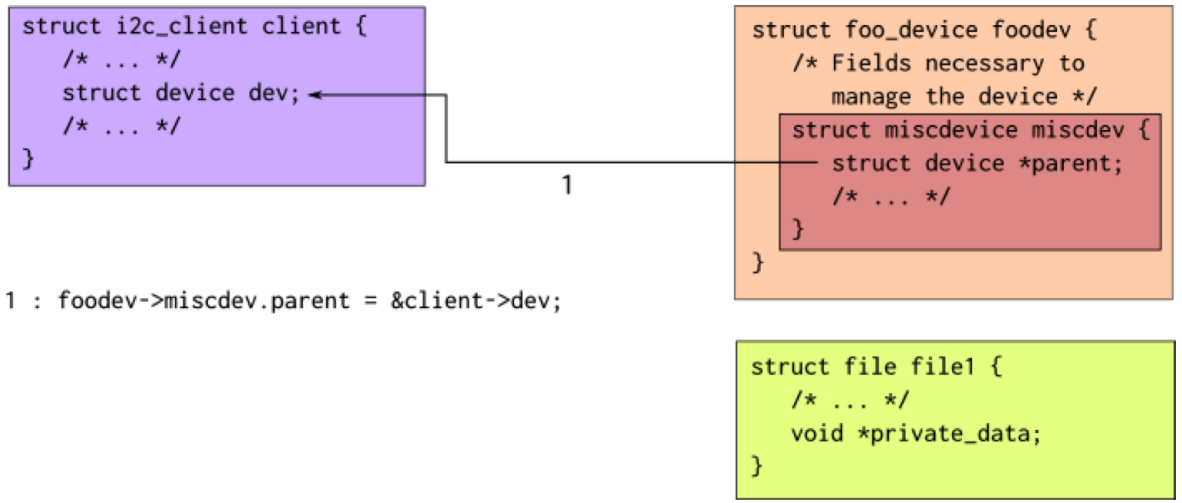
```
struct foo_device foodev {
   /* Fields necessary to
      manage the device */
   struct miscdevice miscdev {
      struct device *parent;
      /* ... */
   }
}
```

```
struct file file1 {
   /* ... */
   void *private_data;
}
```

- How, from one of these structures, to access the others?

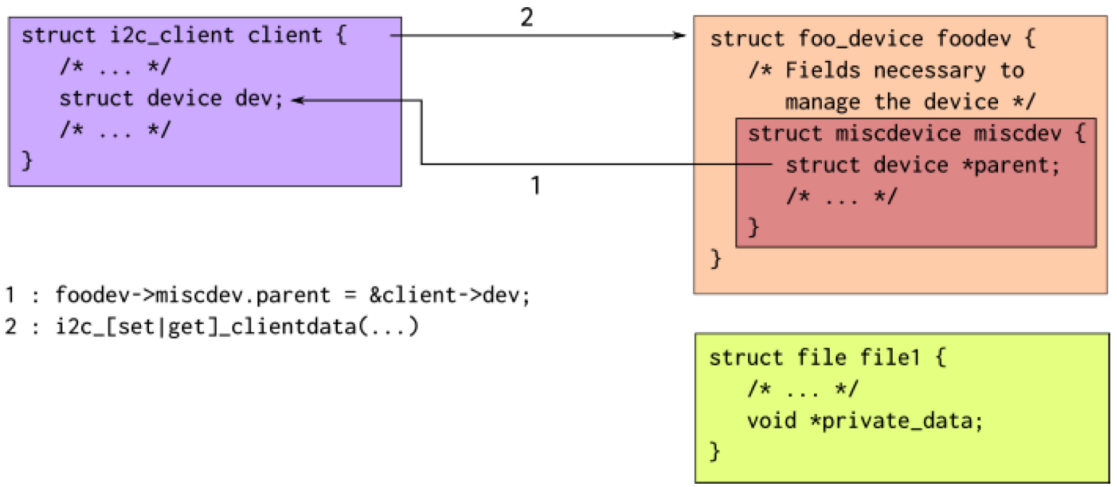
struct miscdevice to struct device

- In the probe function, we initialize the parent field of the struct miscdevice with the address of the dev field of the struct i2c_client



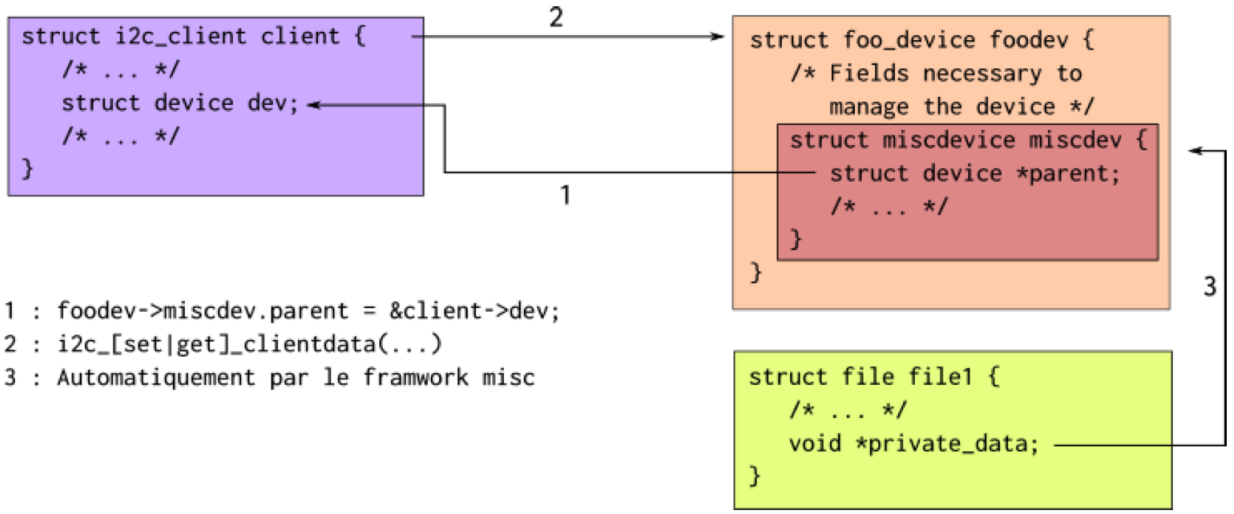
struct i2c_client to struct foo_device

- The i2c_set_clientdata function allows to store any pointer in the i2c_client structure
- This pointer can be retrieved by i2c_get_clientdata
- In the probe function, we use this function to store the address of foodev in client



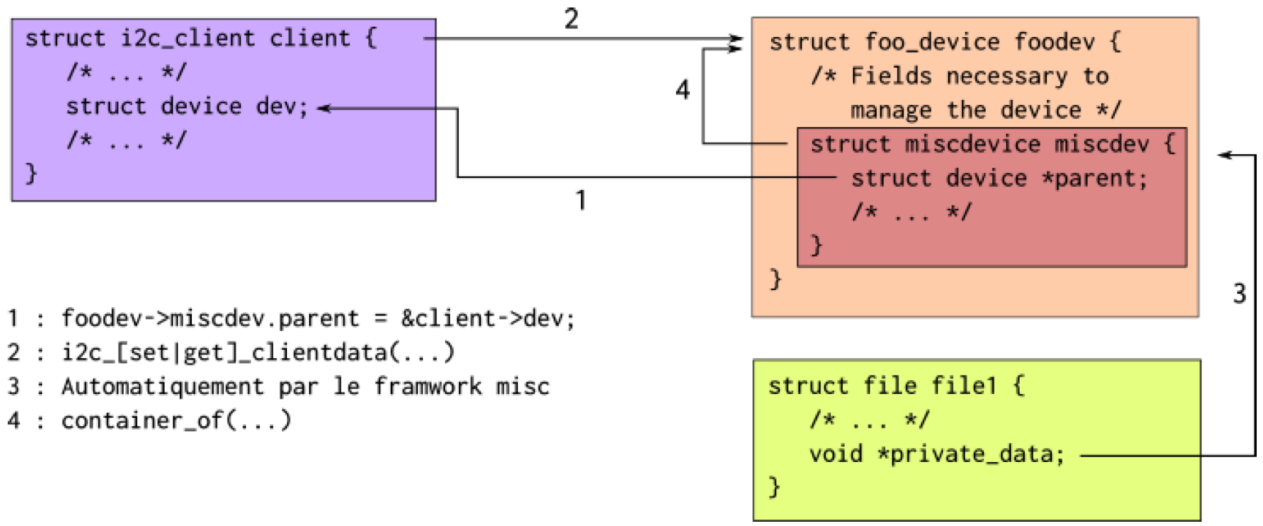
struct file to struct miscdevice

- The misc framework automatically stores in the private_data field of the struct file structure the address of the struct miscdevice structure corresponding to the open file



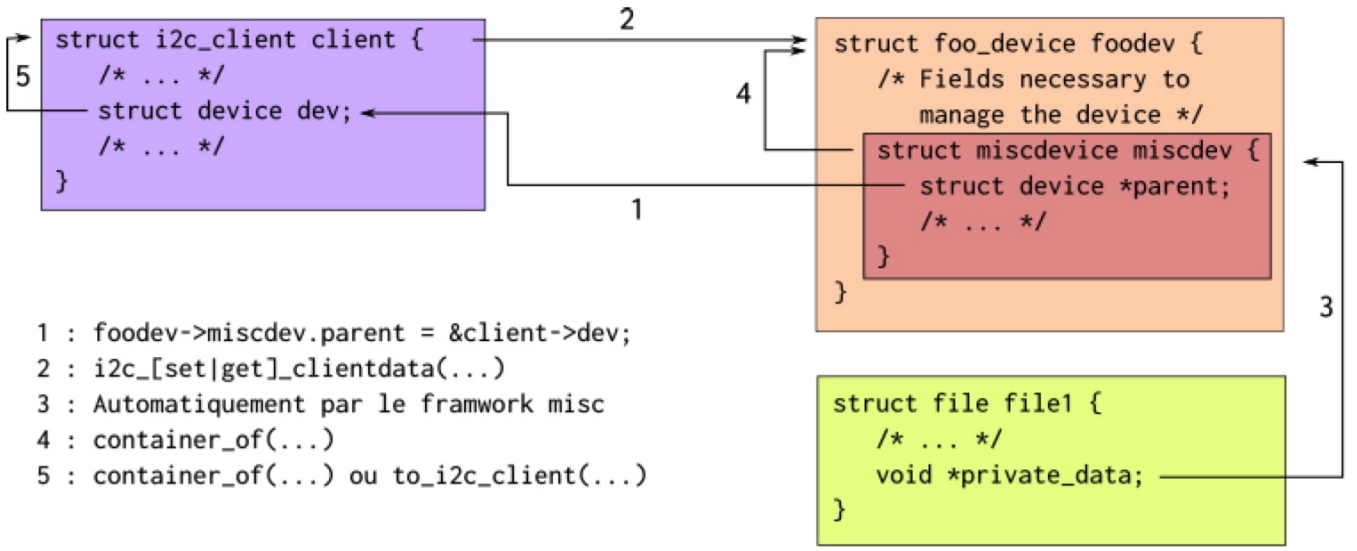
struct miscdevice to struct foo_device

- From the address of the miscdev field of the struct foo_device, it is possible to find the address of the structure that contains it (foodev) using the macro container_of
- ptr_foodev = container_of(ptr, struct foo_device, miscdev)

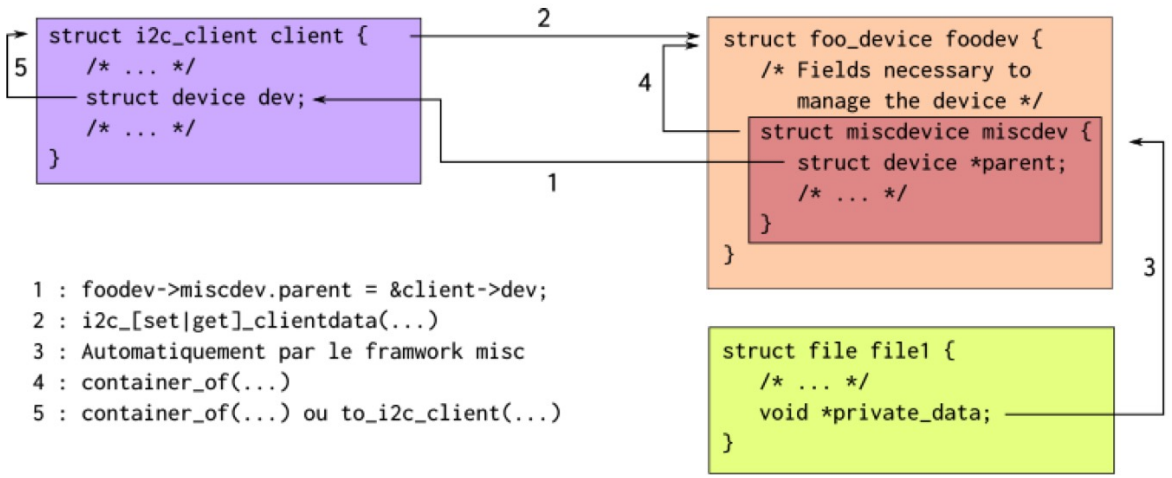


Struct device to struct i2c_client

- We can do the same to go back to client from the address of the dev field using the macro container_of
- You can also use the macro to_i2c_client which is easier to use



Summary



- In the functions receiving a struct file as an argument (the callback functions in the event of an operation on the special file), we can find
  - The structure struct foo_device thanks to 3 and 4
  - The structure struct i2c_client thanks to 3, 1 and 5

Plan

- Introduction
- Classic recall functions
- Data transfer to/from user space
- Frameworks
- Links between the different data structures used
- Conclusion

Conclusion

- We have just seen, for the character devices

  - The main functions to be implemented to react to system calls made by applications on special files

  - The data transfer between the kernel space and user space

  - The links between the main data structures that the structures and driver will have to manipulate

- We still have an important point to see concerning the sleep of a process (which, for example, asks for a blocking read when no data is available for the moment)

Basic API master
Write (master to slave)

```
int i2c_master_send(const struct i2c_client *client ,
const char *buf, int count);
```

- client : structure representing the slave device
- buf : pointer to the buffer containing the data to be transmitted to the slave
- count : number of bytes to transmit
- Returns the number of bytes transmitted or a negative error code

Basic API master
Message Transfer

```
int i2c_transfer(struct i2c_adapter *adap, struct i2c_msg
*msg int num );
```

- adap : structure representing the I2C controller (can be retrieved from the client :

  client->adapter)
- msgs : array of messages num : number of messages
- Returns the number of messages transferred or a negative error code

Basic API master
Message Transfer

```
struct i2c_msg {
        __u16 addr;         /* slave address */
        __u16 flags;
        __u16 len;          /* msg length */
        __u8 *buf;          /* pointer to msg data */
};
#define I2C_M_RD 0x0001     /* read data, from slave to master
```

- addr : slave address (can be retrieved at from the client : client->addr)
- flags : flags
- len : number of data to read or write
- buf : pointer to the buffer containing the data to be written or, in case of writing, where the read data will be stored

Your Turn to Play

- You will write the beginning of a device driver for an I2C accelerometer
- This device is emulated by QEMU